# Done

Welcome Shashank Prakash from Database Architecture, Scale, and NoSQL with Elasticsearch

## To Multi-Master or not to Multi-Master - that is the question

Your organization is reaching the limits of its relational database application as it scales and you are in meeting to decide to move to a multi-master approach or not. Expanding the size of the single-master instance is quite expensive and requires a long-term contract. The multi-master approach is much less expensive and can use commodity hardware. Take a position for or against the switch to multi-master proposal. Write 3-5 paragraphs and share your reasoning on the position you have taken. Feel free to bring prior experience that you might have into the conversation. Feel free to now more than the instructor on this topic :) Let's see what we can learn from talking and listening to each other's perspectives. The primary goal is to have a conversation.

Don't take off points for little mistakes. If they seem to have done the assignment give them full credit. Feel free to make suggestions if there are small mistakes. Please keep your comments positive and useful. If you do not take grading seriously, the instructors may delete your response and you will lose points.

**Please Upload Your Submission:**

Three to five paragraphs on why (or why not) your organization should switch to a multi-master approach

Paragraph

## Overview

The organization I work in is a financial banking system that provides multiple credit services and savings accounts. As such, I am against switching to a multi-master database architecture for this application. While the idea of reducing costs and using commodity hardware may seem attractive, especially in light of growing resource demands, the tradeoffs in terms of data consistency, integrity, and operational complexity make this option unsuitable for such a high-stakes domain.

## Explanation

Banking systems deal with highly sensitive and transactional data—customer balances, credit scores, interest accruals, loan repayments—where strict consistency and data correctness are non-negotiable. A multi-master setup typically involves eventual consistency or complex conflict resolution logic when two or more nodes accept writes. In the financial domain, such conflicts can result in severe consequences like double-spending, overdrafts, or incorrect loan approvals. These are not just technical bugs—they can lead to regulatory violations, financial loss, and loss of customer trust.

Done

Additionally, the operational overhead of multi-master replication—such as detecting and resolving write conflicts, maintaining ordering of transactions, and ensuring transactional integrity across nodes—is significantly more complex than a single-master architecture. Financial institutions are heavily regulated and audited, and systems must support ACID-compliant behavior with robust auditing and traceability. Multi-master systems struggle to deliver on this front without substantial custom engineering and risk management practices in place.

## Alternative

Instead of switching to a multi-master model, the bank should consider read scaling through replicas, vertical scaling where viable, and possibly partitioning by account regions or services with careful boundaries. These strategies can help scale the application while preserving ACID guarantees and without introducing the data anomalies that can arise in a multi-master environment.

## Conclusion

While the idea of cost savings and scalability through a multi-master system is tempting, it is a risky move for a financial institution. The cost of a data integrity failure far outweighs the savings on infrastructure, making it more prudent to maintain strong consistency and consider alternative scaling solutions better aligned with the critical nature of financial services.

Enter optional comments below

As per my experience, there are indeed aspects of financial services as well where a multi-master database may prove to be more useful but banking systems need to strictly adhere to regulations and fairness policies that make it difficult to promote BASE architectures in usual circumstances.

**Submit**

The total number of points for this assignment is 10. Your peers will give you a grade from 0 - 6. You will get 2 for each peer assignment you assess. You need to grade a minimum of 2 peer assignments. You can grade up to 5 peer assignments if you like but grading extra assignment will not increase your score.